

The Hit Zones

Team 063

Abstract

Understanding. Fans want understanding. They want to understand the game of baseball when they watch it, not just out of a desire for enjoyment but also out of a desire to stay informed. While at an MLB game recently, I overheard the gentleman next to me discussing his troubles reading the scoreboard, “there’s so much information up there, I just can’t read it all.” Thinking about what he said, made me realize I share his sentiment to a degree. At games I do see dozens of numbers and stats littered across the scoreboards, which while useful and often necessary, can also act as information overload at times.

With this dynamic in mind, I sought to create a simple, yet effective graphic that tells us something about the type of hitter up to bat. A graphic that would be displayed on the scoreboard for each batter throughout the course of a game. This is where the hit zones come into play, splitting the field into three regions and showing where a batter has historically hit the ball. The idea is to provide quicker and digestible visual context for the fans as to the type of hitter they are watching. Is it a pull hitter? Do they enjoy some oppo tacos from time to time? Do their hit zones indicate a more balanced approach at the plate? These are the questions I seek to shed light on.

Table of Contents

Background	4
Data	6
Analysis	8
Discussion	11
Acknowledgements	13
Works Cited	14
Appendices	15

Background

When it comes to visualizing batted ball data, there have been a variety of different use cases and implementations. Such examples can be found at 6-4-3 Charts, a company that specializes in creating interactive player tools that help bring visuals to life. As you can see in the image below, they created an [All Sprays Tool](#) that allows scouts and teams to better understand which region of the field a batter hits into under which types of events (6-4-3 Charts 2026).

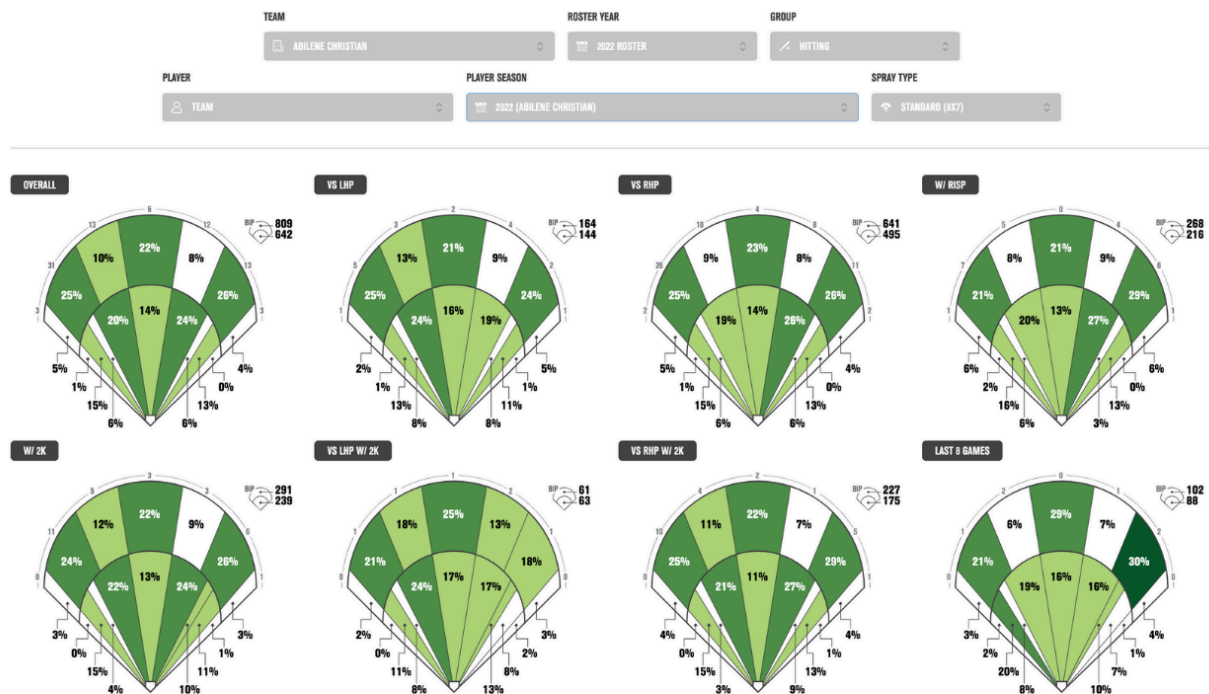
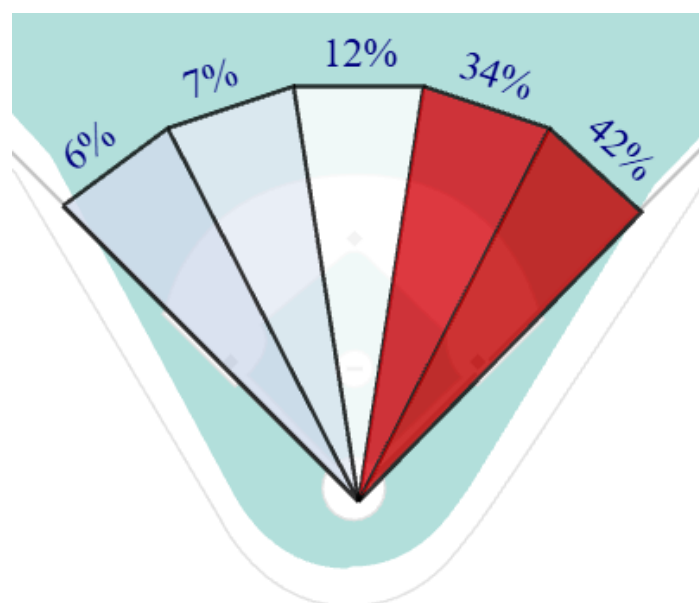


Figure 1. All Sprays interactive tool from the website at 6-3-4 Charts

Another inspiration for this project was Baseball Savant's [Infield Slice Chart](#) tool, which also splits the field into smaller slices that each highlight the percentage of batted balls hit into the respective slice (MLB Advanced Media 2026). However, the limitation of this tool is that it only provides the zones for batted balls that have a distance of 200 feet or less, hence not for any balls hit into the outfield or home runs.



Shohei Ohtani - LHB

Figure 2. Infield Slice Chart interactive tool from the website at Baseball Savant

My goal for this project is to combine the best of both these worlds by simplifying the scoreboard visualization while also accounting for all batted balls hit into play. In order to also prioritize the engagement element and ease of interpretation for fans, I simplified the visual into only three hit zones (left, center, right) and did not distinguish between infield and outfield zones as shown in the first diagram. While adding several zones and different layers to the diagram is useful for a more advanced look at a player's batted ball profile, I also recognize that the fans are the main audience for this year's project theme. A visual that takes too long to analyze or is too granular to absorb easily is not going to be very appealing to them when the player comes up to bat. In addition, knowing that this could be displayed on an outfield scoreboard that the majority of fans will be somewhat far away from, I figured reducing the number of zones allows them to quickly receive a high level view of the type of batter they are seeing.

Data

As part of this project, we were given anonymized player and ball tracking data from a season of pre-Covid minor league games, as well as other event and lineup data to work with. For me, the goal with the data at first was to get to a table for each player that included all of their fair batted balls and their corresponding hit zones from the season of data provided, as well as relevant context for each observation such as the game and team info, game state, and player IDs.

After working through a few combinations of the data (see Appendix B for more info), I was able to whittle down the table of interest to only balls acquired in the field of play. Essentially, the location of where the ball was fielded, which I considered a solid estimate of the region of the field in which the ball was hit. The table ended up showing 41 batted balls in that game, which makes sense intuitively when you consider the fact that a game potentially has 54 outs to work with.

Now that I had the filtered ball events table ready to go, I was able to combine that table with the balls positions and lineups table mentioned earlier. Once cleaned, this resulted in a table with 41 observations and 16 columns, with each valid batted ball included as well as its corresponding context (the position of the batted ball, the inning, the pitcher and batter IDs, and more). From here, I labeled the hit zones as 1, 2, and 3, with 1 corresponding to the left field region, 2 corresponding to the center field region, and 3 corresponding to the right field region. To properly distinguish the zones from each other, I had to calculate the hit angles for each batted ball and then filter them into their proper hit zones. The left and right field foul lines are angled at 135 and 45 degrees respectively, so I defined the cutoffs for each zone as 135 to 105 degrees for zone 1, 105 to 75 degrees for zone 2, and 75 to 45 degrees for zone 3. Once the hit angles

were calculated and the cutoffs established, each batted ball was filtered into its corresponding zone (shortened snippet with only 8 columns shown below).

	game_string	play_per_game	batter	ball_position_x	ball_position_y	ball_position_z	hit_angles	hit_zone
0	y1_d061_VKA_PHD	2	VKA-5565	29.32599	147.6795	4.234620	78.768396	2
1	y1_d061_VKA_PHD	3	VKA-7844	78.28440	149.4936	0.951447	62.360573	3
2	y1_d061_VKA_PHD	8	VKA-7206	-34.96410	191.6787	4.106190	100.337663	2
3	y1_d061_VKA_PHD	11	PHD-8619	37.59990	381.3900	0.000000	84.369605	2
4	y1_d061_VKA_PHD	22	PHD-7166	-197.38080	271.2804	0.000000	126.039247	1

Table 1. Valid batted balls table that now include the angle and zone of the batted ball. The 'VKA-5565' label at the top represents the complete player ID, with the 3 letters serving as the team code and the 4 numbers serving as the player code. The same goes for every player in the data.

Eventually, I did try to repeat this process on all the data provided, but certain operations ended up expanding the data even further and it became way too slow to work with. However, with this filtered table above complete for this game (which, after removing batted balls outside of the hit zones, resulted in 39 valid batted balls), I was then able to use this template to create a couple functions for further analysis (see Appendices F and G).

Analysis

In order to analyze the players' hit zones, I decided to create a few functions that would generate and further filter each players' valid batted balls over the course of the given season's data. There are two main functions I want to discuss. The first function, the hit zone filter, allows the user to enter in the batter ID, as well as a number of different context parameters, such as game ID, inning number, pitcher ID, runners on base, and more. This function serves as a way to obtain a table containing all the players' valid batted balls fitting the context provided by the user. The second function, hit zone percentage generator, builds on the first one by acting as a summary function that converts those filtered batted balls to historical hit zone percentages and visualizes a spray chart of the batted balls labeled within the three hit zones. For example, in terms of potential use-cases, a scoreboard operator could input the player ID or name as well as

any additional context parameters and receive a row of data containing the percentage of batted balls that player has hit into each of the respective zones and the total number of batted balls within the context given. They would also be able to see an attached spray chart that contains each of the batted balls labeled according to their zone, which serves as a helper visual (more info about these sections in Appendices E, F, and G). The operator could then take those historical percentages and input them into their operating system to be displayed onto the scoreboards live in the stadium (Figure 3 below shows as an example of the function result). Alternatively, a dashboard could be set up within the operating room that allows for the operators to simply input a batter’s name with the desired context and then receive the corresponding hit zones (Figure 4), which would then be projected live to the stadium scoreboards.

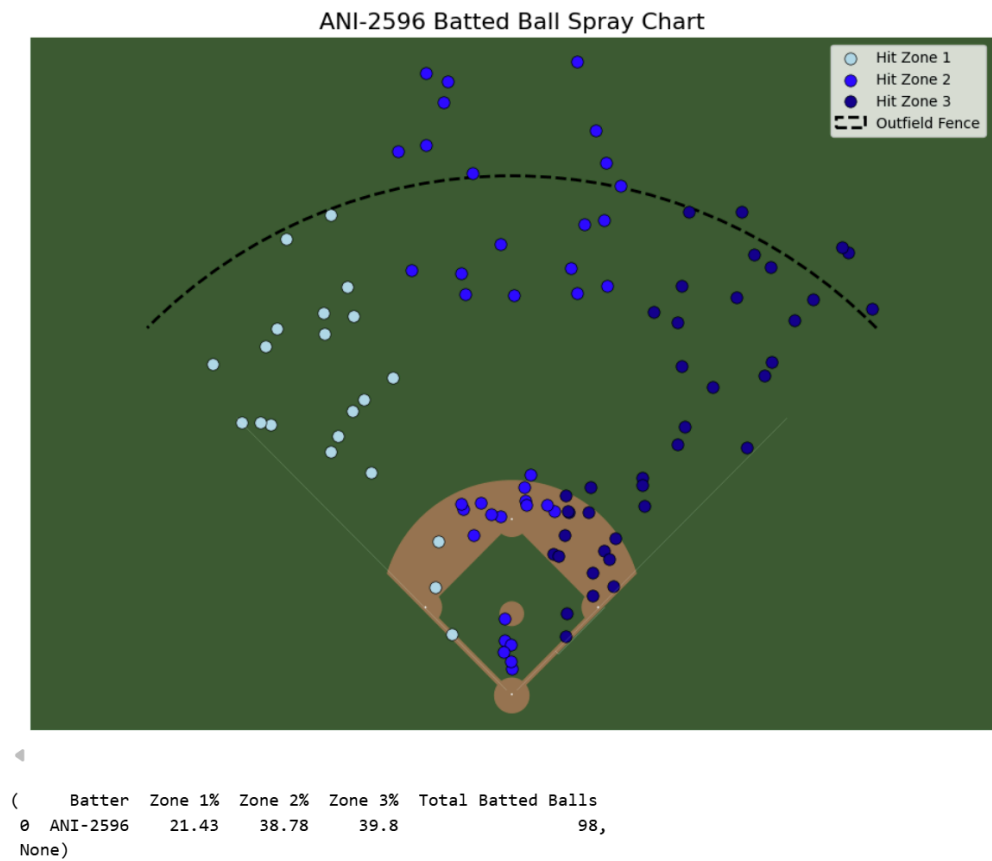


Figure 3. Example of the hit zone percentage generator function showing the batter’s historical batted ball data, as well as their hit zone labeled spray chart.

I also decided to visualize the final concept of the hit zone, in terms of what it would look for the player shown above and then what it could look like on a real scoreboard one day. I used Adobe InDesign and Photoshop, in order to get a crisper, more polished look at it. As you can see, the darker the shade of the zone, the larger the percentage of batted balls in that zone. The visual is also designed to be catered to the batting team's scoreboard colors. For example, the light shades of blue in Figure 4 would be used in neutral settings (used here since I don't know the colors of the team that player 'ANI-2596' plays for) or with teams that share that scoreboard color scheme. In the case of Ketel Marte in Figure 5, his hit zones match the color scheme of the Diamondbacks scoreboard. This visual could also be sponsored by a company, and in this case I used Netflix, but the home team could use whatever company they're working with that chooses to sponsor it.

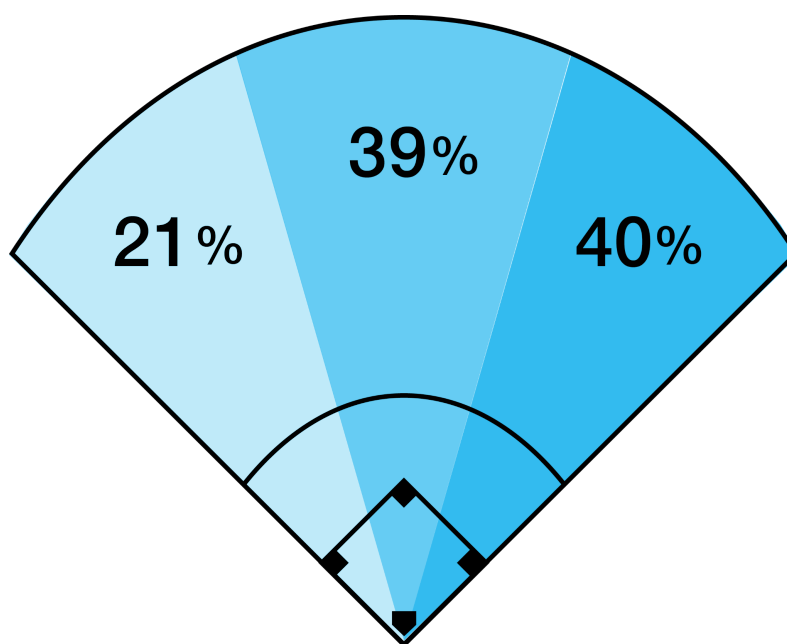


Figure 4. This is what the final minimal hit zones would look for the player 'ANI-2596'. I also decided that only whole numbers would be shown on the visual for simplicity, so any decimals would be rounded up or down.



Figure 5. Ideal use case of hit zones being displayed on a stadium scoreboard (original image - [AZ_game](#)).



Figure 6. GIF showing progression of visual: [team-063-scoreboard-visual-progression](#). Alternative link: [GIF visual](#).

Discussion

This visual was developed out of a desire to give the fans more context about the game that is both digestible and informative, essentially giving them a visual cue as to something that has happened and can happen within the game. My main aim is to give context as to the type of hitter up to bat and their batted ball tendencies so that someone viewing the batter can get a sense of where they are historically likely to hit the ball at that given moment and context within the game. Hopefully this helps fans better understand, from a big-picture perspective, the different types of hitters and how game-state can influence where they hit the ball.

There are a number of ways in which this project can be further applied and developed within front offices and stadiums across MLB. While the visuals I have shown would ideally stay up for the duration of the at-bat, scoreboard operators could choose to show the visual with the players' percentages for each count state, clutch situation, specific handedness matchup, etc. For example, if it is a high leverage at-bat in the 8th inning, the operator could show the batter's hit zones for each time the batter has come to the plate in the 8th inning with their team down by 1 run facing a lefty with 2 strikes and so on and so forth. A sample size cutoff for when the visual should be shown could also be implemented, maybe somewhere around a minimum of 10-15 batted balls in order to display the graphic, maybe more or less, it depends on the team or operator's preferences. In terms of design, hit zones can also be catered to the dimensions of the field the batter is playing and an option could be provided to increase the number of hit zones displayed (maybe 5 hit zones instead of 3).

I decided against modeling due to the inherent difficulty of trying to predict where a hitter will hit the ball within a given at-bat, coupled with a struggle to identify clear features I could derive from the data given. This project could be enhanced by adding predictive modeling and

turning the historical percentages shown in the hit zones into predicted percentages highlighting where the batter is most likely to hit the ball. It would also be important that the hit zones are updated throughout the game so that a players' hit zones in their second at-bat reflect the location of the batted ball from their first at-bat. Coaches and scouts could utilize hit zones within their scouting reports, proprietary dashboards, and websites to help inform player evaluations and future development, as well as add on launch angle, exit velocity, and other filtering parameters. Additionally, this could be developed into a Streamlit web app where users can enter the batter's name with the additional context mentioned throughout this paper and see the generated hit zones for that batter right away.

Acknowledgments

I want to acknowledge everyone who has helped contribute to the development and adaptation of this project. Big thanks to Billy Fryer for his consistent and timely advice as well as his shared tutorial code, to Meredith Wills for motivating me to finish strong and discussing this project with me from a conceptual standpoint, and to Cameron Adams for his writing and messaging tips. I also want to thank Paul Alvord, a good family friend, for sharing his design wisdom to help me arrive at the visual and narrative concept for the hit zones. Thank you to everyone who shared websites and tips on the Discord as well, those helped even when I didn't know I needed them. Last, but not least, I want to thank my parents for helping my idea come to life through their constant inspiration and for helping me stay on track and motivated to keep working towards the completion of this project.

Works Cited

- Brannigan, Liam. "Cheatsheet for Pandas to Polars | Rho Signal." *Cheatsheet for Pandas to Polars*, Rho Signal, 7 Nov. 2024, www.rhosignal.com/posts/polars-pandas-cheatsheet/.
- Drucker, Ross. "Sportypy — Sportypy Documentation." *Sportypy*, 2022, sportypy.sportsdataverse.org/.
- "Interactive Stats – 6–4–3 Charts." *6-4-3 Charts*, 643charts.com/web-application/.
- "MLA Citations: Overview - Purdue Owl® - Purdue University." *Purdue Owl*, Purdue University College of Liberal Arts, owl.purdue.edu/owl/research_and_citation/mla_style/mla_formatting_and_style_guide/mla_formatting_and_style_guide.html.
- Newman, Tod. "A Quick Tour of a Baseball Park Scoreboard." *TodNewman.Com*, todnewman.com/a-quick-tour-of-a-baseball-park-scoreboard/.
- "Polar Grid In Degrees With Radius 1." *ClipArt ETC*, Educational Technology Clearinghouse, 2004, etc.usf.edu/clipart/43000/43012/polar_24-11_43012.htm.
- "Python - Numpy Arctan2 Bug or Usage Issues? - Stack Overflow." *Stack Overflow*, Stack Exchange Inc, 2012, stackoverflow.com/questions/13308639/numpy-arctan2-bug-or-usage-issues.
- "Shohei Ohtani Stats: Statcast, Visuals & Advanced Metrics | Baseballsavant.com." *Baseball Savant*, MLB Advanced Media, baseballsavant.mlb.com/savant-player/shohei-ohtani-660271.

Appendices

Appendix A

Attached here is an image showing some of my original visual concepts for this project and how I kind of worked through different versions. Obviously, these didn't end up being what I used, but they helped inform me about how I wanted to tweak the hit zones for my final rendition within the paper.

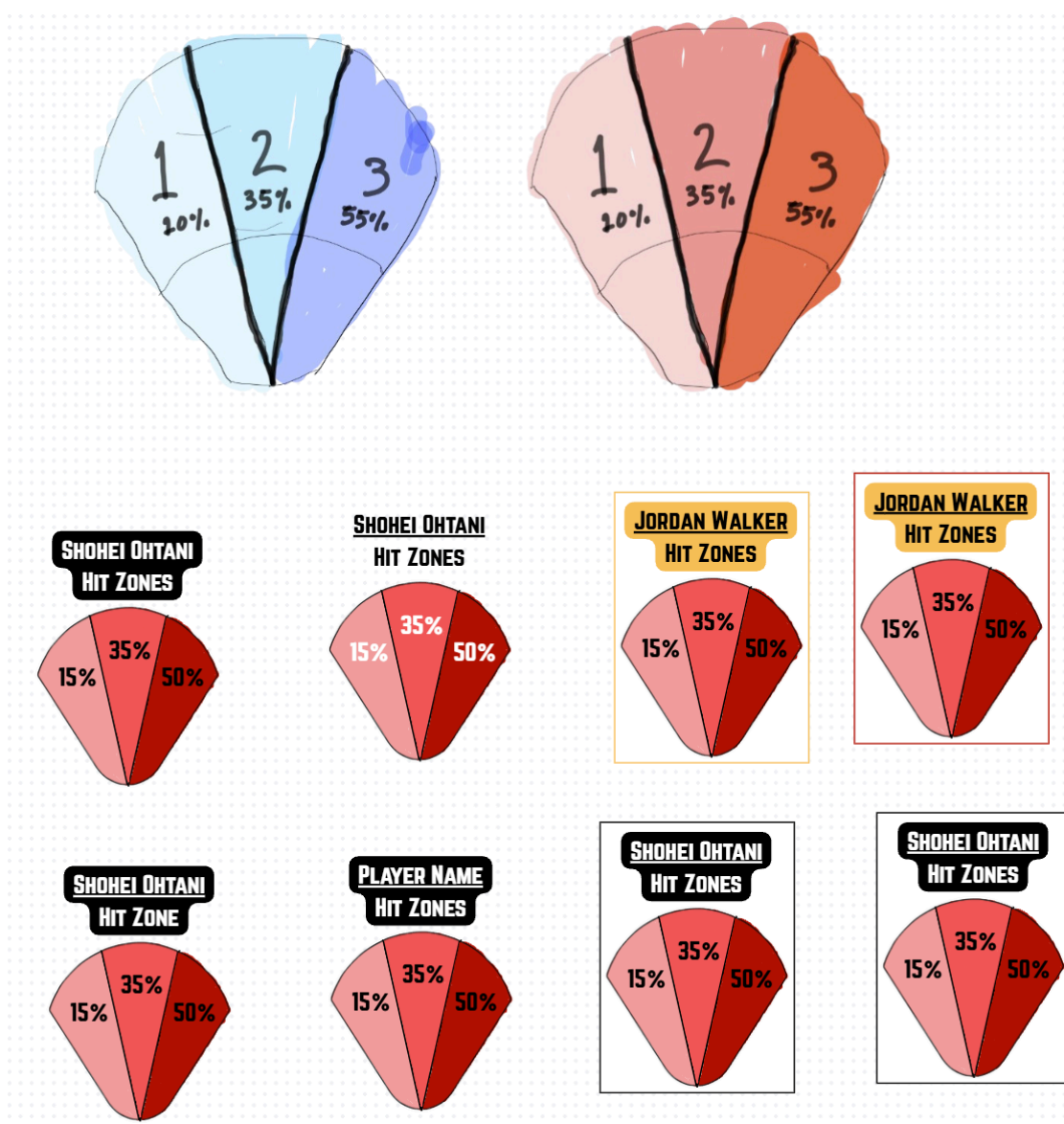


Figure 7. Earlier versions showcasing the visual evolution of the hit zones.

Appendix B

To start the table generation process, I began with only using data from a single game in order to test out the table generation on a smaller sample size. I first combined the ball positions data from that game with the lineups data, which includes the IDs for each player on the field, baserunner IDs, and other relevant team info. This resulted in a table with about 30 columns and over 6000 observations, which was now ready to be joined later with my filtered ball events table. In order to find those valid batted ball locations, I decided that a fair strategy was to use the ball events data to filter down to the point where the ball was acquired after being hit into play. Using the data from this game, I first needed to remove any ball events that were simply the ball bouncing, like the ball bouncing on the field before getting into the fielder's glove. I then created two smaller ball event tables, one that was filtered for only balls acquired by fielders after being hit into the field of play and another that was filtered for home runs. From there, I combined these two tables into one ball events table that included all the valid batted balls for that game with foul balls removed (table is shown below).

	game_string	play_per_game	hit_timestamp	player_id	ball_eventcode
0	y1_d061_VKA_PHD	2	78414	4	2
1	y1_d061_VKA_PHD	3	112164	4	2
2	y1_d061_VKA_PHD	8	220314	6	2
3	y1_d061_VKA_PHD	11	398014	8	2
4	y1_d061_VKA_PHD	22	723064	7	2

Table 2. Valid batted ball table that includes game info, player IDs, and ball event codes (event codes are explained more fully in Appendix B). The player IDs represent different positions in the field and the ball event codes represent different ball events. For example the first observation was the second play of the game and it involved a batted ball being acquired by a second baseman.

Appendix C

The filtering logic used on the ball events tables discussed in the data section was implemented using Polars dataframes and checking for certain event codes. For example, with the balls acquired table, the event codes had to be reverse-engineered in order to filter for all the valid balls acquired. As you can see in the code snippet below, I had to first filter for ball event codes equal to 2, which indicated a ball being acquired by a fielder, while also making sure it was a fielded ball that came after a batted ball, signified by event codes equal to 4.

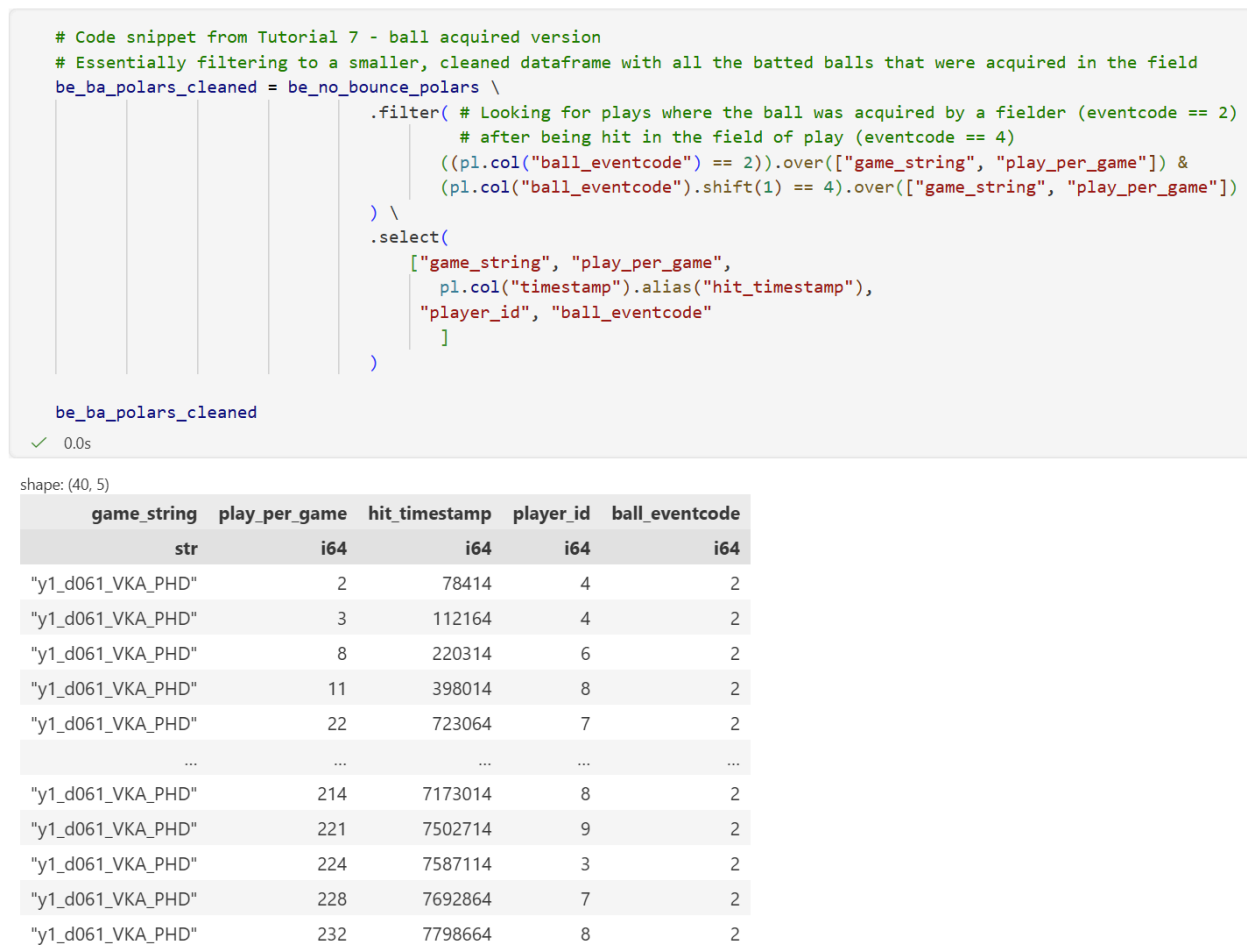


Figure 8. Code snippet detailing filtering logic used to generate the balls acquired dataframe.

The same type of logic was applied when trying to find all batted balls that ended up being homeruns, but this time I had to filter for ball event codes equal to 11, indicating a homerun being hit.

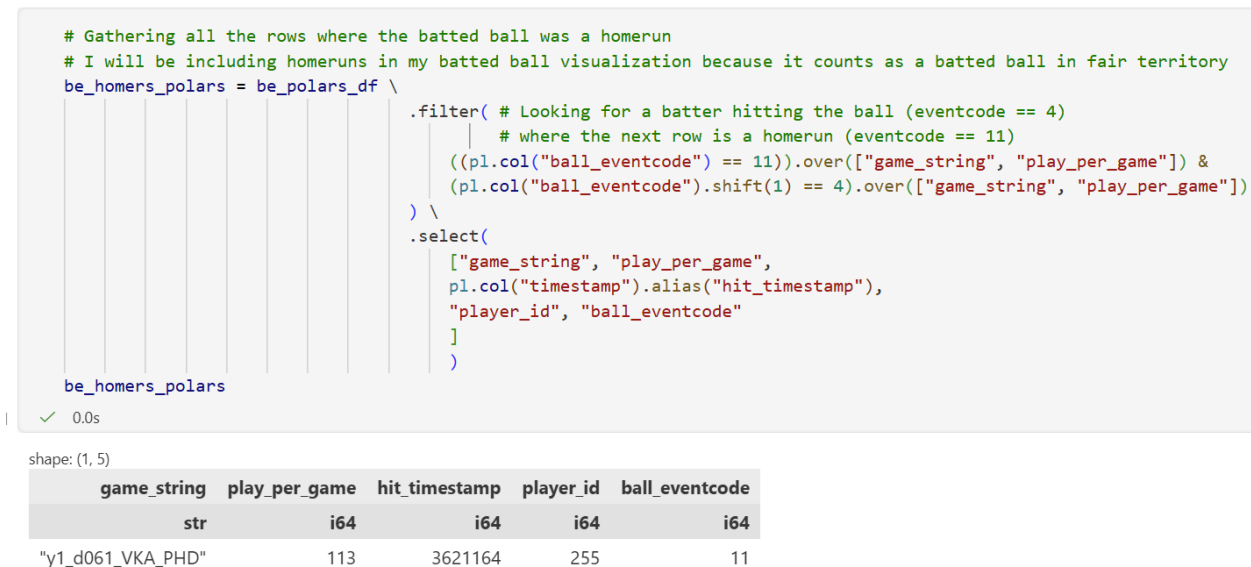


Figure 9. Code snippet detailing filtering logic used to generate the homeruns dataframe.

Finally, after filtering, I was able to generate the table containing all valid batted balls for the given game by simply combining the balls acquired and homerun observations into one single table.



Figure 10. Code snippet showing the first few rows of resulting valid batted balls dataframe.

Appendix D

When it came to creating hit zones within the tables, I had to utilize a Polars grid in order to differentiate between the foul lines correctly. For reference, the minor league field lines span between angles of 45 degrees and 135 degrees, with 45 representing the right foul line and 135 representing the left foul line.

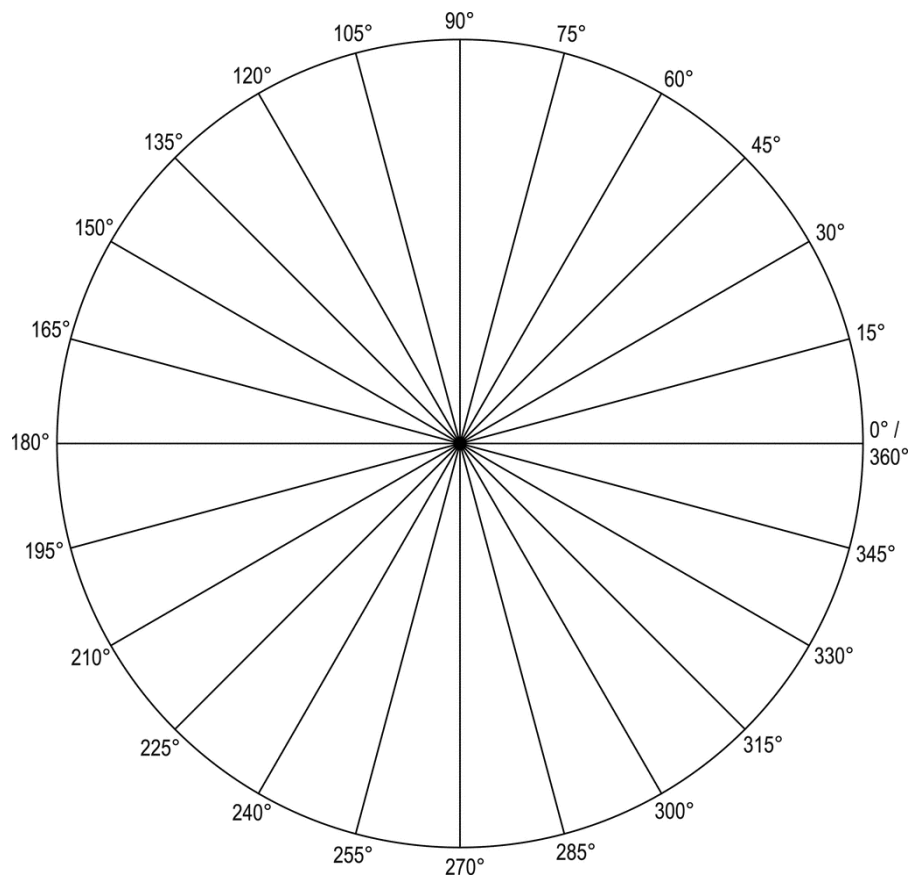


Figure 11. Polar grid shown in degrees with radius 1.

Once I defined the angles to split on, I had to calculate them for each batted ball in order to filter each observation into its corresponding hit zone. The hit angles were calculated using an arctan function that uses the given x and y coordinates to come up with the resulting angle, and were then converted to degrees using the outside numpy function seen in the code below (Stack Overflow).

```

# Labeling the field zones for filtering the batted balls
# (3 corresponds to right field, 2 to center field, and 1 to left field)
field_zones = ['3', '2', '1']

# Defining the angle cutoffs for each zone to differentiate between left, center, and right field
degree_zones = [45.0, 75.0, 105.0, 135.0]

# Calculating the hit angles and adding that resulting data as an additional column
main_df['hit_angles'] = np.degrees(np.arctan2(main_df['ball_position_y'], main_df['ball_position_x']))
main_df.head()

# Adding a column corresponding to the zone in which the batted ball was hit
main_df['hit_zone'] = pd.cut(main_df['hit_angles'], bins = degree_zones,
                             labels = field_zones, include_lowest=True)

```

Figure 12. Code detailing the hit angle and hit zone calculations.

Appendix E

For visualizing the batted ball spray chart, I created a function, the hit zone visualizer, that uses the MiLBField() object from the Sportypy library and filters the batted ball dataframe by hit zone in order to visualize each one in a different color. I then used restricted scatter plots to make sure each zone's batted balls were being visualized within their proper distance ranges.

```
def hit_zone_visualizer(player_table, batter):
    # Generating sample minor league , field
    sample_field = MiLBField()
    fig, ax = plt.subplots(figsize=(12, 12))
    sample_field.draw(ax=ax, display_range='full')

    # Adding outfield grass
    ax.add_patch(Rectangle(
        xy=(-500, -50),
        width=900,
        height=900,
        facecolor="#395d33",
    ))

    zone_one = player_table[player_table['hit_zone'] == '1']
    zone_two = player_table[player_table['hit_zone'] == '2']
    zone_three = player_table[player_table['hit_zone'] == '3']

    # Visualizing sample player's batted balls
    ax.scatter(zone_one['ball_position_x'], zone_one['ball_position_y'], color='lightblue',
               edgecolor='black', linewidth = 0.5, s=67, zorder=15, label='Hit Zone 1')

    ax.scatter(zone_two['ball_position_x'], zone_two['ball_position_y'], color='blue',
               edgecolor='black', linewidth = 0.5, s=67, zorder=15, label='Hit Zone 2')

    ax.scatter(zone_three['ball_position_x'], zone_three['ball_position_y'], color='darkblue',
               edgecolor='black', linewidth = 0.5, s=67, zorder=15, label='Hit Zone 3')

    # Adding imaginary outfield fence with 375 foot dimensions,
    # not exactly real life accurate but works for this verification
    fence = Arc((0, 0), width=750, height=750, angle=0, theta1=45, theta2=135,
               color='black', linewidth=2, linestyle = '--', zorder=10, label = 'Outfield Fence')
    ax.add_patch(fence)

    # Title of chart
    ax.set_title(f'{batter} Batted Ball Spray Chart', fontsize=16)
    ax.set_aspect('equal')

    ax.set_xlim(-350, 350)
    ax.set_ylim(-25, 475)
    ax.legend()
    plt.show()
```

Figure 13. Code showing the details behind the batted ball spray chart visualization.

Appendix F

The hit zone filter function was created in order to take in several user arguments and filter the batted ball dataframe accordingly. You can see in the code snippet below that each function parameter was also set to None by default so that it wouldn't break if the user decided not to include a certain parameter, such as the pitcher faced or the number of runners on base when filtering the data.

```
def hit_zone_filter(batter, game = None, inning = None, pitcher = None,
                   on_1b = None, on_2b = None, on_3b = None, runners_on = None,
                   is_homerun = None, hit_zone = None):
    """
    This function generates the zone of the field in which the ball was hit
    for each of the given batter's batted balls. It also takes in additional
    optional context, such as game_string, inning, pitcher faced, etc.

    Required parameters:
    batter (string) - the player id code of the batter

    Optional parameters:
    game (string) - the game_string in which the batted ball occurred
    inning (int) - the inning in which the batted ball occurred
    pitcher (string) - the pitcher off which the batted ball was hit
    on_1b, on_2b, on_3b (boolean) - True or False values with each indicating whether or not a batter is on the given base
    runners_on (int) - the total numbers of runners on base during the batted ball
    hit_zone (category - input argument as string) - the region of the field in which the ball was hit
    is_homerun (boolean) - True or False value indicating whether or not the batted ball was a homerun
    """
```

Figure 14. Code showing the details behind the hit_zone_filter function parameters.

In order to check if the user did enter data for any of the parameters shown, I used a series of statements on the filtered hit zone dataframe.

```

# Checking to make sure that a proper batter ID is provided
if batter == '':
    return 'Please provide Batter ID.'

filtered_hit_zone = hit_zone_generator(batter)

# The below if-else statements look like a lot, but they are all
# just checking to see if a value was provided for the specific parameter
if game is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['game_string'] == game]

if inning is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['inning'] == inning]

if pitcher is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['pitcher'] == pitcher]

if on_1b is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['on_1b'] == on_1b]

if on_2b is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['on_2b'] == on_2b]

if on_3b is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['on_3b'] == on_3b]

if runners_on is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['ROB'] == runners_on]

if is_homerun is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['is_homerun'] == is_homerun]

if hit_zone is not None:
    filtered_hit_zone = filtered_hit_zone[filtered_hit_zone['hit_zone'] == hit_zone]

# Returning filtered hit zone dataframe
return filtered_hit_zone

```

Figure 15. Code showing the details behind the hit_zone_filter function if statements.

The hit zone generator function seen in Figure 13, was another helper function I first created in order to filter the original batted balls by the given player. Essentially, it goes through the entire dataframe cleaning process as well as the hit zone generation process again on the

given player in order to return all accurate batted balls for that player over the course of the season. You can see the implementation of this function in the figures below.

```
def hit_zone_generator(batter):
    """
    This function generates the zone of the field in which the ball was hit
    for each of the given batter's batted balls.

    Parameters: batter (string) - the player id code of the batter
    """

    # Checking to make sure that a proper batter ID is provided
    if batter == '':
        return 'Please provide Batter ID.'

    # Filtering the complete lineups dataframe to only lineups that include the given batter
    lineups_w_batter = lineups_all_games_pandas[lineups_all_games_pandas['batter'] == batter]

    # Dropping rows with null values, only a small amount of the dataset, much less than 1%, as mentioned earlier in the notebook
    bp_all_games_cleaned = bp_all_games_pandas.dropna()

    # Filtering complete ball positions dataframe down to only games that the batter hit in
    bp_batter_games = bp_all_games_cleaned[bp_all_games_cleaned['game_string'].isin(lineups_w_batter['game_string'])]

    # Converting the play_per_game and timestamp columns from floats (decimals) to ints (integers)
    bp_batter_games['play_per_game'] = bp_batter_games['play_per_game'].astype(int)
    bp_batter_games['timestamp'] = bp_batter_games['timestamp'].astype(int)

    valid_ball_events_batter = valid_ball_events_pandas.copy()

    # Merging the complete valid ball events dataframe (which was created earlier) with the games the batter hit in
    # This will provide a dataframe that includes all the hitter's valid batted balls with the ball positions included
    be_bp_batter = valid_ball_events_batter.merge(bp_batter_games, on='timestamp', how='inner')

    # Removing unnecessary columns and renaming some columns
    be_bp_batter_cleaned = be_bp_batter.drop(
        columns=['game_string_y', 'home_team', 'away_team',
                 'year', 'day', 'play_per_game_y']).rename(
        columns={'game_string_x': 'game_string',
                 'play_per_game_x': 'play_per_game'})

    # Merging the lineup info from games the batter hit in with the valid ball position dataframe from above
    be_bp_w_batter = lineups_w_batter.merge(be_bp_batter_cleaned, on=['game_string', 'play_per_game'], how='inner')
```

Figure 16. Code showing the details behind the hit_zone_generator function.

```

# Removing unnecessary columns
be_bp_w_batter_demo_cleaned = be_bp_w_batter.drop(
    columns=['home_team', 'away_team', 'is_pickoff', 'catcher',
            'first_base', 'second_base', 'third_base', 'shortstop',
            'left_fielder', 'center_fielder', 'right_fielder'])

final_batter_df = be_bp_w_batter_demo_cleaned.copy()

# Labeling the field zones for filtering the batted balls (3 corresponds to right field, 2 to center field, and 1 to left field)
field_zones = ['3', '2', '1']

# Defining the angle cutoffs for each zone, essentially differentiating between left, center, and right field
degree_zones = [45.0, 75.0, 105.0, 135.0]

# Calculating the hit angles and adding that resulting data as an additional column
final_batter_df['hit_angles'] = np.degrees(np.arctan2(final_batter_df['ball_position_y'], final_batter_df['ball_position_x']))

# Adding a column corresponding to the zone in which the batted ball was hit
final_batter_df['hit_zone'] = pd.cut(final_batter_df['hit_angles'], bins = degree_zones,
    labels = field_zones, include_lowest=True)

# Essentially removing foul balls here (balls to the left of 3rd base line or to the right of 1st base line) and dropping duplicate rows if they exist
final_batter_df = final_batter_df.dropna()
final_batter_df = final_batter_df.drop_duplicates()

# Replacing the existing runners on base columns with T/F indicators showing whether or not a runner is on each base during the batter ball
final_batter_df['on_1b'] = final_batter_df['on_1b'] != 'NA'
final_batter_df['on_2b'] = final_batter_df['on_2b'] != 'NA'
final_batter_df['on_3b'] = final_batter_df['on_3b'] != 'NA'

# Adding a new column that represents the total number of runners on base during each batted ball
final_batter_df['ROB'] = final_batter_df[['on_1b', 'on_2b', 'on_3b']].eq(True).sum(axis=1)

# Adding a new column that indicates whether or not a batted ball was a homerun
final_batter_df['is_homerun'] = final_batter_df['ball_eventcode'] == 11

return final_batter_df

```

Figure 17. Code showing the details behind the hit_zone_generator function.

Appendix G

The hit zone percentage generator is the main function that builds on all the other helper functions. This function, like the hit zone filter, takes in the same user arguments, but this time calculates the batter's historical percentages for each batted ball within the 3 different hit zones. It utilizes the hit zone filter and hit zone visualizer functions in order to process the user arguments and apply them correctly, and returns the historical percentages along with the batted ball spray chart for that batter.

```
def hit_zone_percentage_generator(batter, game = None, inning = None, pitcher = None,
    on_1b = None, on_2b = None, on_3b = None, runners_on = None, is_homerun = None):
    """
    This function generates the historical batted ball hit zone percentages for each batter.

    Required parameters:
    batter (string) - the player id code of the batter

    Optional parameters:
    game (string) - the game_string in which the batted ball occurred
    inning (int) - the inning in which the batted ball occurred
    pitcher (string) - the pitcher off which the batted ball was hit
    on_1b, on_2b, on_3b (boolean) - True or False values with each indicating whether or not a batter is on the given base
    runners_on (int) - the total numbers of runners on base during the batted ball
    is_homerun (boolean) - True or False value indicating whether or not the batted ball was a homerun
    """
```

```

# Checking to make sure that a proper batter ID is provided
if batter == '':
    return 'Please provide Batter ID.'

# Setting up filtered dataframe from filter function based on user inputs
batter_hit_zone = hit_zone_filter(batter, game, inning, pitcher,
                                  on_1b, on_2b, on_3b, runners_on, is_homerun)

# Calculating the size of the filtered dataframe to get total number of batted ball events
hz_all_balls = len(batter_hit_zone)

# Checking to make sure division by zero does not occur
if hz_all_balls == 0:
    hz_one_percentage = 0.0
    hz_two_percentage = 0.0
    hz_three_percentage = 0.0
else:
    hz_one_total = (batter_hit_zone['hit_zone'] == '1').sum()
    hz_two_total = (batter_hit_zone['hit_zone'] == '2').sum()
    hz_three_total = (batter_hit_zone['hit_zone'] == '3').sum()

    hz_one_percentage = ((hz_one_total / hz_all_balls) * 100).round(2)
    hz_two_percentage = ((hz_two_total / hz_all_balls) * 100).round(2)
    hz_three_percentage = ((hz_three_total / hz_all_balls) * 100).round(2)

data = {
    "Batter": [batter],
    "Zone 1%": [hz_one_percentage],
    "Zone 2%": [hz_two_percentage],
    "Zone 3%": [hz_three_percentage],
    "Total Batted Balls": [hz_all_balls]
}

batter_percentages = pd.DataFrame(data)
return batter_percentages, hit_zone_visualizer(batter_hit_zone, batter)

```

Figure 18. Code showing the details behind the `hit_zone_percentage_generator` function.